EXPT NO: 4	A PYTHON PROGRAM TO PERFORM PERT ANALYSIS
DATE:	

AIM:

To perform Program Evaluation Review Technique (PERT) Analysis for project scheduling and management using Python.

PROCEDURE:

1. INSTALL REQUIRED LIBRARIES:

- Make sure you have pandas, networkx, matplotlib, and numpy installed.
- You can install them using pip if not already installed.

  -bash

  -Copy code

  -pip install pandas networkx matplotlib numpy

2. DEFINE THE PROBLEM:

- Identify the tasks, their optimistic, most likely, and pessimistic durations, and dependencies.

3. SET UP THE PROBLEM IN PYTHON:

- Use NetworkX to represent the project tasks and their dependencies.
- Calculate the expected duration and variance for each task.
- Use topological sort to identify the critical path and perform PERT analysis.

4. RUN THE CODE:

- Execute the Python code to get the critical path, project duration, and associated uncertainty.

5. ANALYZE THE OUTPUT:

- Interpret the results and validate the solution against the project schedule.

PROBLEM 1: SOLVE THE FOLLOWING PROBLEM USING PERT ANLAYSIS

Activity	Predecessors	Optimistic Time (O)	Most Likely Time (M)	Pessimistic Time (P)
A	-	1	2	3
B	A	2	4	6
C	A	1	3	5
D	B, C	3	4	5

CODE

import networkx as nx import matplotlib.pyplot as plt

class PERTTask: def **init**(self, name, o, m, p): self.name = name self.o = o self.m = m self.p = p self.duration = (o + 4 * m + p) / 6

```
self.early_start = 0
 self.early_finish = 0
 self.late_start = float('inf')
 self.late_finish = float('inf')

 self.successors = []
```

class PERT: def **init**(self, tasks, dependencies): self.tasks = {name: PERTTask(name, *time) for name, time in tasks.items()} self.dependencies = dependencies self.build_graph()

def build_graph(self):
for task, deps in self.dependencies.items():
for dep in deps:

self.tasks[dep].successors.append(self.tasks[task])

def forward_pass(self):
# Initialize start tasks
for task in self.tasks.values():
if not self.dependencies[task.name]:
task.early_start = 0
task.early_finish = task.duration

# Process tasks
for task in self.tasks.values():

```
for succ in task.successors:
        succ.early_start = max(succ.early_start, task.early_finish)
        succ.early_finish = succ.early_start + succ.duration

def backward_pass(self):
    max_finish = max(task.early_finish for task in self.tasks.values())

    # Initialize end tasks
    for task in self.tasks.values():
        if not task.successors:
            task.late_finish = max_finish
            task.late_start = task.late_finish - task.duration

    # Process backward
    for task in reversed(list(self.tasks.values())):
        for succ in task.successors:
            task.late_finish = min(task.late_finish, succ.late_start)
            task.late_start = task.late_finish - task.duration

def find_critical_path(self):
    return [
        task.name for task in self.tasks.values()
        if round(task.early_start, 2) == round(task.late_start, 2)
    ]

def run(self):
    self.forward_pass()
    self.backward_pass()
    return self.find_critical_path()

def visualize_network(self):
    G = nx.DiGraph()

    for task in self.tasks.values():
        G.add_node(
            task.name,
            label=f"{task.name}\nES:{round(task.early_start,2)},
EF:{round(task.early_finish,2)}\nLS:{round(task.late_start,2)},   LF:{round(task.late_finish,2)}"
        )
        for succ in task.successors:
            G.add_edge(task.name, succ.name)
```

```
    pos = nx.spring_layout(G)
    labels = {node: data['label'] for node, data in G.nodes(data=True)}

    plt.figure(figsize=(10,  6))
    nx.draw(G, pos, with_labels=True, node_size=3000,
            node_color="lightgreen", edge_color="gray", font_size=10)
    nx.draw_networkx_labels(G, pos, labels, font_size=8)

    plt.title("PERT Network Diagram")
    plt.show()


if name == "main": tasks = { 'A': (1, 2, 3), 'B': (2, 4, 6), 'C': (1, 3, 5), 'D': (3, 4, 5) }

dependencies = {
    'A': [],
    'B': ['A'],
    'C': ['A'],
    'D': ['B', 'C']
}

pert = PERT(tasks, dependencies)
critical_path = pert.run()

print("Critical Path:", " -> ".join(critical_path))
```

OUTPUT

Critical Path: A -> B -> D

PROBLEM 2: SOLVE THE FOLLOWING PROBLEM USING PERT ANLAYSIS

Task	(o, m, p)
A	(2, 3, 4)
B	(1, 2, 3)
C	(2, 4, 6)
D	(3, 5, 7)
E	(2, 3, 8)

CODE

```
import networkx as nx import matplotlib.pyplot as plt

class PERTTask: def init(self, name, o, m, p): self.name = name self.o = o self.m = m self.p = p
self.duration = (o + 4 * m + p) / 6

    self.early_start = 0
    self.early_finish = 0
    self.late_start = float('inf')
    self.late_finish = float('inf')

    self.successors = []


class PERT: def init(self, tasks, dependencies): self.tasks = {name: PERTTask(name, *time) for name, time
in tasks.items()} self.dependencies = dependencies self.build_graph()

def build_graph(self):
    for task, deps in self.dependencies.items():
  for dep in deps:
            self.tasks[dep].successors.append(self.tasks[task])

def forward_pass(self):
    for task in self.tasks.values():
        if not self.dependencies[task.name]:
            task.early_start = 0
            task.early_finish = task.duration

    for task in self.tasks.values():
        for succ in task.successors:
            succ.early_start = max(succ.early_start, task.early_finish)
            succ.early_finish = succ.early_start + succ.duration

def backward_pass(self):
    max_finish = max(task.early_finish for task in self.tasks.values())

    for task in self.tasks.values():
        if not task.successors:
            task.late_finish = max_finish
            task.late_start = task.late_finish - task.duration
```

```
    for task in reversed(list(self.tasks.values())):
        for succ in task.successors:
            task.late_finish = min(task.late_finish, succ.late_start)
            task.late_start = task.late_finish - task.duration

def find_critical_path(self):
    return [
        task.name for task in self.tasks.values()
        if round(task.early_start, 2) == round(task.late_start, 2)
    ]

def run(self):
    self.forward_pass()
    self.backward_pass()
    return self.find_critical_path()

def visualize_network(self):
    G = nx.DiGraph()

    for task in self.tasks.values():
        G.add_node(
            task.name,
            label=f"{task.name}\nES:{round(task.early_start,2)},
EF:{round(task.early_finish,2)}\nLS:{round(task.late_start,2)},   LF:{round(task.late_finish,2)}"
        )
        for succ in task.successors:
            G.add_edge(task.name,  succ.name)

    pos = nx.spring_layout(G)
    labels = {node: data['label'] for node, data in G.nodes(data=True)}

    plt.figure(figsize=(10,  6))
    nx.draw(G, pos, with_labels=True, node_size=3000,
            node_color="lightgreen", edge_color="gray", font_size=10)
    nx.draw_networkx_labels(G, pos, labels, font_size=8)

    plt.title("PERT Network Diagram")
    plt.show()


if name == "main": tasks = { 'A': (2, 3, 4), 'B': (1, 2, 3), 'C': (2, 4, 6), 'D': (3, 5, 7), 'E': (2, 3, 8) }
```

```
dependencies = {
    'A': [],
    'B': ['A'],
    'C': ['A'],
    'D': ['B'],
    'E': ['C', 'D']
}

pert = PERT(tasks, dependencies)
critical_path = pert.run()

print("Critical Path:", " -> ".join(critical_path))
```

OUTPUT

Critical Path: A -> B -> D -> E

RESULT

Thus Program Evaluation Review Technique (PERT) Analysis for project scheduling and management using Python has been successfully implemented.